

M17 / LEARNER BOOK

Architecture and engineering decisions

Standalone Zero-to-Job-Ready Data Engineer 2026

10 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE | How to use this module 3

DE17-01 | From requirements to architecture 4

DE17-02 | Batch vs streaming decisions 8

DE17-03 | Warehouse vs lakehouse 11

DE17-04 | Managed service vs build yourself 14

DE17-05 | Scaling and bottleneck reasoning 17

DE17-06 | Reliability and failure domains 20

DE17-07 | Cost-aware architecture 23

DE17-08 | Security and governance by design 26

DE17-09 | Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes 30

DE17-10 | Architecture review and defense 34

[START HERE](#)

How to use this module

Open the matching practice folder from the course root: 16_M17/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

Module outcome. By the end you can turn requirements into a defensible architecture, choose batch/stream and warehouse/lakehouse boundaries, reason about managed-vs-build, bottlenecks, failure domains, cost, security/governance, and explain promotion technologies such as CDC, data contracts, Vault, data mesh and Kubernetes at the right level.

DE17-01

From requirements to architecture

Watch first

[1. Microsoft Learn - Apply Well-Architected Framework to your solution design](#)

02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[2. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 five categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 1 of 10 | Primary practice:

16_M17/practice/architecture_patterns/01_requirements_to_architecture.py

Why this matters

Architecture starts before a service name. A good design turns business outcomes and non-functional requirements into explicit data contracts, latency targets, security boundaries, recovery goals and cost constraints. If you jump straight from 'we need a dashboard' to a list of products, the diagram may look professional while answering the wrong problem.

Picture it in your head

Picture a bridge engineer. Before choosing steel thickness, they need span, traffic, wind, maintenance access and acceptable failure behavior. Data architecture is the same translation problem: requirements become forces on the design.

Words before we use them

New word	Meaning before we use it
functional requirement	What the system must do for users or downstream systems.
non-functional requirement	How well or under what constraints the system must operate, such as latency, reliability, security or cost.
SLA/SLO	A formal or internal service target used to reason about acceptable performance/reliability.
constraint	A boundary the design must respect, such as budget, region, team skill or compliance.
trade-off	A deliberate gain in one property that usually costs something elsewhere.

Teacher explanation from first principles

Start with six questions: who consumes the data, what decision/action depends on it, how fresh it must be, how wrong it may be before harm occurs, what data is sensitive, and what failure/recovery window is acceptable. Then add volume and growth. Only after these are explicit should you compare architecture options. Separate facts from guesses: '20 million events/day today' is evidence; 'will probably double' is an assumption that should be tested with a scenario rather than treated as truth.

A second architecture-review lens

A reviewer should be able to point to every box and ask which requirement forced it to exist. If the answer is only 'best practice', the design is not yet grounded. Separate hard constraints from preferences: residency law or RPO zero may be hard; a preferred cloud service may be negotiable. Also record revisit triggers, such as volume crossing 5x, latency tightening below one minute, or the team gaining platform support. Architecture is a current decision under stated conditions, not a permanent truth.

Useful video / source route

[VIDEO: Microsoft Learn - Apply Well-Architected Framework to your solution design - Open video](#)

Watch: 02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 five categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: arch](#)

[OFFICIAL/CURRENT: waf](#)

Use rule: Watch/read for the mental model or current interface, then return to this lesson and its practice. External media is support; the architecture decision and evidence must be yours.

Follow with me once

A retailer wants hourly store sales by 07:00 local time, can tolerate 30 minutes of delay during recovery, must keep cardholder data out of analytics, and has a two-person data team. Translate this into: batch ingestion every 15 minutes or hourly, curated sales grain, PII tokenization before analytics, a published freshness SLO, replayable raw evidence, simple managed components, and an alert if reconciliation or freshness fails. Notice that the requirements already rule out some unnecessarily complex designs.

What you should see / why it worked

An architecture decision sheet linking each important requirement to a design response and a measurable proof. Another engineer should be able to disagree with your choice without first guessing your assumptions. The important part is the trace from requirement to decision to evidence; the same product name can be correct in one requirement set and wrong in another.

Break it: a realistic failure

A team selects Kafka, Spark, Kubernetes and a lakehouse because these technologies appear in job descriptions. The workload is one 2 GB nightly file with next-morning reporting. The result creates 24x7 operating cost and a steep on-call burden without a requirement that needs streaming or cluster orchestration.

Reason through a realistic incident

Preserve the evidence first. State the violated requirement, then identify the smallest failing boundary. Separate business-data state from compute/configuration/identity state. Bound the blast radius, choose a reversible repair, validate the original requirement, and finally test a changed case. Architecture recovery should reduce uncertainty, not hide it behind more components.

Try it yourself

Use the supplied clinic-reporting scenario. Write functional requirements, at least six non-functional requirements, known constraints, assumptions, and three questions you still need from stakeholders. Sketch a first architecture only after those are written.

Fresh changed retry

Change the requirement from 'next morning' to 'fraud alert within 10 seconds' and explain exactly which architecture boundaries must change and which can remain batch.

Validation and recovery ladder

- PASS only when your decision cites a requirement/constraint and an independent control or measurement.
- If a fact is unknown, label it as an assumption and state what you would measure before committing.
- Reject a design that solves the happy path but has no replay/recovery, ownership, security boundary or cost story.
- After repair/review, change latency, volume, budget or compliance and prove you can adapt instead of reciting the
- same diagram.

Revision in your own words

From requirements to architecture is a decision problem. Remember the requirement, the competing options, the trade-off, the failure boundary and the evidence that would validate the choice.

Teach it back

Explain why architecture is primarily a requirements-to-trade-offs exercise rather than a cloud-service memorization exercise.

Evidence to keep

Save your first architecture/scorecard, reviewer challenge, corrected decision, changed-case retry, and a short ADR explaining why you chose one option over another. Do not erase the first design when you improve it.

DE17-02

Batch vs streaming decisions

Watch first

[1. Confluent Developer - Stream Processing | Apache Kafka 101 \(2025 Edition\)](#)

00:00-04:10 stream processing and batch/stream comparison

[2. Microsoft Learn - Apply Well-Architected Framework to your solution design](#)

02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 2 of 10 | Primary practice:

16_M17/practice/architecture_patterns/02_batch_streaming_decision.py

Why this matters

Streaming is valuable when action loses value quickly, but it adds continuous state, ordering, replay, capacity and operational concerns. Batch is not 'old technology'; it is often the safest choice when latency requirements permit it. The architecture skill is choosing the simplest model that meets the decision deadline.

Picture it in your head

Mail delivery versus a live phone call. Both move information. A monthly statement does not need a live call; emergency dispatch does. The delivery mode should match the consequence of waiting.

Words before we use them

New word	Meaning before we use it
decision latency	How long the business can wait before data must influence an action.
event time	When the business event actually happened.
processing time	When the system handled the event.
replay	Reprocessing retained events after failure or logic correction.
hybrid	An architecture intentionally combining streaming and batch paths.

Teacher explanation from first principles

Use a decision tree instead of fashion. Ask: what is the maximum useful delay? Is the source naturally event-driven? Must individual events trigger actions, or are periodic aggregates enough? Can downstream consumers handle continuous updates? What replay and state semantics are required? A 5-minute dashboard might still be micro-batch. A stream can also feed a durable batch/lakehouse path for reconciliation and reprocessing.

A second architecture-review lens

The hardest part is often not choosing batch or stream, but defining the consistency boundary between them. A streaming alert can be provisional while an hourly reconciliation becomes the accounting truth. That is acceptable when consumers know the contract. Conversely, two paths that silently compute the same metric differently create permanent reconciliation debt. If you design a hybrid path, name which system owns final truth, how late corrections flow, and how replay avoids double effects.

Useful video / source route

[VIDEO: Confluent Developer - Stream Processing | Apache Kafka 101 \(2025 Edition\) - Open video](#)

Watch: 00:00-04:10 stream processing and batch/stream comparison

[VIDEO: Microsoft Learn - Apply Well-Architected Framework to your solution design - Open video](#)

Watch: 02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch/read for the mental model or current interface, then return to this lesson and its practice. External media is support; the architecture decision and evidence must be yours.

Follow with me once

Compare three workloads. Payroll: monthly, correctness dominates latency - batch. Card fraud: suspicious transaction must be acted on in seconds - streaming event path plus durable replay. Executive daily revenue: source events may arrive continuously, but a governed hourly/daily aggregate can remain batch or micro-batch. The same company can rationally use all three.

What you should see / why it worked

A latency/value table and a justified choice that names state, replay, ordering and operational implications - not merely 'streaming is faster'. The important part is the trace from requirement to decision to evidence; the same product name can be correct in one requirement set and wrong in another.

Break it: a realistic failure

A product team asks for 'real time' because the phrase sounds modern. No one can identify a user action that changes within an hour. The stream becomes a permanent on-call system while reports are opened once each morning.

Reason through a realistic incident

Preserve the evidence first. State the violated requirement, then identify the smallest failing boundary. Separate business-data state from compute/configuration/identity state. Bound the blast radius, choose a reversible repair, validate the original requirement, and finally test a changed case. Architecture recovery should reduce uncertainty, not hide it behind more components.

Try it yourself

Classify six supplied workloads as batch, micro-batch, streaming or hybrid. For each, give the decision deadline, replay expectation and one failure mode.

Fresh changed retry

Take one batch choice and tighten its useful latency by 100x. Re-evaluate source, storage, compute and monitoring without automatically assuming every component must become streaming.

Validation and recovery ladder

- PASS only when your decision cites a requirement/constraint and an independent control or measurement.
- If a fact is unknown, label it as an assumption and state what you would measure before committing.
- Reject a design that solves the happy path but has no replay/recovery, ownership, security boundary or cost story.
- After repair/review, change latency, volume, budget or compliance and prove you can adapt instead of reciting the
- same diagram.

Revision in your own words

Batch vs streaming decisions is a decision problem. Remember the requirement, the competing options, the trade-off, the failure boundary and the evidence that would validate the choice.

Teach it back

Give one case where a streaming source still feeds a batch serving layer and explain why that is not contradictory.

Evidence to keep

Save your first architecture/scorecard, reviewer challenge, corrected decision, changed-case retry, and a short ADR explaining why you chose one option over another. Do not erase the first design when you improve it.

DE17-03

Warehouse vs lakehouse

Watch first

[1. Microsoft Learn - Apply Well-Architected Framework to your solution design](#)

02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 3 of 10 | Primary practice:

16_M17/practice/architecture_patterns/03_warehouse_lakehouse_decision.py

Why this matters

Warehouse and lakehouse are not team identities; they are serving/storage patterns with different strengths. A warehouse often gives strong governed SQL analytics and operational simplicity. A lakehouse brings open-file/object-storage economics with table semantics and multi-engine flexibility. Many production platforms use both patterns or products that blur the boundary.

Picture it in your head

Think of a highly organized library versus a warehouse-sized archive with a catalog and table system layered over it. The right question is what kinds of readers, formats, transformations and governance you need - not which label sounds newer.

Words before we use them

New word	Meaning before we use it
warehouse	Analytical database pattern optimized for governed SQL/BI workloads.
lakehouse	Architecture combining lake/object-storage openness with table-management/warehouse-like capabilities.
open table format	Metadata/protocol layer that gives files table semantics such as versions and transactions.
serving layer	Data representation designed for a consumer/query pattern.
workload isolation	Keeping one type of workload from unpredictably harming another.

Teacher explanation from first principles

Compare on workload and operating model: BI concurrency, semi/unstructured data, open engine access, ML/Spark needs, governance, cost model, performance predictability and team skills. Do not treat 'cheap storage' as 'cheap system': query engines, compaction, cataloging and operational labor matter. Likewise, do not assume a warehouse cannot ingest files or a lakehouse cannot serve SQL; modern products overlap.

A second architecture-review lens

Think in data products and workload boundaries rather than brand categories. Raw evidence may live cheaply in object storage while governed serving data uses an engine optimized for concurrent SQL. The same Delta/Iceberg-style table may be queried by multiple engines, yet each query engine still has capacity, security and cost behavior. Your decision should include who owns table maintenance, catalog policy, semantic definitions and performance expectations; storage format alone does not solve those responsibilities.

Useful video / source route

[VIDEO: Microsoft Learn - Apply Well-Architected Framework to your solution design - Open video](#)

Watch: 02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[OFFICIAL/CURRENT: dbx](#)

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch/read for the mental model or current interface, then return to this lesson and its practice. External media is support; the architecture decision and evidence must be yours.

Follow with me once

A finance reporting team needs highly governed SQL, stable semantic models and hundreds of BI queries. A data science team needs raw/semi-structured clickstream and Spark. One defensible architecture lands durable source data in object storage/lakehouse tables, curates reusable silver data, and serves governed finance marts through a warehouse/SQL serving surface. The design explains the boundary instead of forcing one label everywhere.

What you should see / why it worked

A comparison matrix that connects consumers and data shapes to a storage/serving choice and states what will remain intentionally outside that platform. The important part is the trace from requirement to decision to evidence; the same product name can be correct in one requirement set and wrong in another.

Break it: a realistic failure

A team migrates all BI tables into a lake because storage is cheaper but does not plan compaction, catalog permissions, concurrency or semantic serving. File cost falls while analyst experience and operations degrade.

Reason through a realistic incident

Preserve the evidence first. State the violated requirement, then identify the smallest failing boundary. Separate business-data state from compute/configuration/identity state. Bound the blast radius, choose a reversible repair, validate the original requirement, and finally test a changed case. Architecture recovery should reduce uncertainty, not hide it behind more components.

Try it yourself

Use the supplied media-company scenario. Decide whether raw events, curated events, finance marts and feature data belong in warehouse, lakehouse or both. Explain ownership and query paths.

Fresh changed retry

Remove Spark/ML from the scenario and make 95% of demand governed BI. Re-evaluate whether the lakehouse remains central or becomes a landing/archive component.

Validation and recovery ladder

- PASS only when your decision cites a requirement/constraint and an independent control or measurement.
- If a fact is unknown, label it as an assumption and state what you would measure before committing.
- Reject a design that solves the happy path but has no replay/recovery, ownership, security boundary or cost story.
- After repair/review, change latency, volume, budget or compliance and prove you can adapt instead of reciting the
- same diagram.

Revision in your own words

Warehouse vs lakehouse is a decision problem. Remember the requirement, the competing options, the trade-off, the failure boundary and the evidence that would validate the choice.

Teach it back

Why is 'warehouse versus lakehouse' often a boundary-placement decision rather than a winner-takes-all decision?

Evidence to keep

Save your first architecture/scorecard, reviewer challenge, corrected decision, changed-case retry, and a short ADR explaining why you chose one option over another. Do not erase the first design when you improve it.

DE17-04

Managed service vs build yourself

Watch first

[1. Microsoft Learn - Apply Well-Architected Framework to your solution design](#)

02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[2. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 five categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 4 of 10 | Primary practice:

16_M17/practice/architecture_patterns/04_managed_vs_build_scorecard.py

Why this matters

Every custom component creates ownership: upgrades, security patches, scaling, backups, observability and specialist knowledge. Managed services trade some flexibility/control for reduced undifferentiated operations. The architecture question is where custom control creates enough business value to justify that ownership.

Picture it in your head

Owning a power generator gives control but also fuel, maintenance and failure responsibility. Buying electricity from the grid gives less control over internals but removes most generator operations. Neither is always right.

Words before we use them

New word	Meaning before we use it
managed service	Platform where a provider operates significant infrastructure lifecycle responsibilities.
control plane	Management layer that configures/coordinates system behavior.
operational burden	Ongoing human/system effort to keep a component secure, available and updated.
lock-in	Cost or difficulty of moving due to APIs, data formats, skills or operational dependencies.
undifferentiated heavy lifting	Work required to operate technology but not unique to the business product.

Teacher explanation from first principles

Score choices on required control, compliance, portability, team expertise, uptime responsibility, upgrade burden, integration, cost and exit plan. 'Managed is expensive' ignores the labor and risk of self-operation. 'Managed is easy' ignores quotas, platform constraints and provider-specific semantics. The best choice often manages commodity layers and keeps business-differentiating logic portable.

A second architecture-review lens

A useful total-ownership review includes at least five hidden jobs: upgrades, security response, backups/recovery, observability/on-call, and capacity planning. Put a named team beside each job. If no team owns it, self-hosting has an unpriced risk. Managed services also need an exit strategy: identify portable data formats, export/replay boundaries and provider-specific features you would have to replace. This turns vague lock-in fear into a concrete migration-cost discussion.

Useful video / source route

[VIDEO: Microsoft Learn - Apply Well-Architected Framework to your solution design - Open video](#)

Watch: 02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 five categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: arch](#)

[OFFICIAL/CURRENT: waf](#)

Use rule: Watch/read for the mental model or current interface, then return to this lesson and its practice. External media is support; the architecture decision and evidence must be yours.

Follow with me once

A six-person analytics team needs orchestration. Self-hosting a scheduler on Kubernetes gives control but adds cluster/scheduler upgrades, HA, secrets, logging and on-call. A managed orchestration service may cost more per visible unit yet reduce total ownership. If the company already has a platform team operating Kubernetes safely, the balance may change.

What you should see / why it worked

A weighted scorecard with assumptions and an explicit owner for the operational responsibilities that remain after the choice. The important part is the trace from requirement to decision to evidence; the same product name can be correct in one requirement set and wrong in another.

Break it: a realistic failure

The team self-hosts because the open-source software is free. Six months later one engineer spends most of the week on upgrades and broken credentials. License cost was zero; total cost of ownership was not.

Reason through a realistic incident

Preserve the evidence first. State the violated requirement, then identify the smallest failing boundary. Separate business-data state from compute/configuration/identity state. Bound the blast radius, choose a reversible repair, validate the original requirement, and finally test a changed case. Architecture recovery should reduce uncertainty, not hide it behind more components.

Try it yourself

Compare managed Kafka versus self-operated Kafka for a team of four. Include availability, patching, security, scaling, portability, cost visibility and incident ownership.

Fresh changed retry

Change the organization to a regulated platform team with 30 experienced SREs and a hard requirement for custom network/security controls. Re-score rather than keeping your first answer.

Validation and recovery ladder

- PASS only when your decision cites a requirement/constraint and an independent control or measurement.
- If a fact is unknown, label it as an assumption and state what you would measure before committing.
- Reject a design that solves the happy path but has no replay/recovery, ownership, security boundary or cost story.
- After repair/review, change latency, volume, budget or compliance and prove you can adapt instead of reciting the
- same diagram.

Revision in your own words

Managed service vs build yourself is a decision problem. Remember the requirement, the competing options, the trade-off, the failure boundary and the evidence that would validate the choice.

Teach it back

What responsibilities disappear with a managed service, and which responsibilities always remain yours?

Evidence to keep

Save your first architecture/scorecard, reviewer challenge, corrected decision, changed-case retry, and a short ADR explaining why you chose one option over another. Do not erase the first design when you improve it.

DE17-05

Scaling and bottleneck reasoning

Watch first

[1. Microsoft Learn - Apply Well-Architected Framework to your solution design](#)

02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 5 of 10 | Primary practice:

16_M17/practice/architecture_patterns/05_bottleneck_capacity_reasoning.py

Why this matters

Scaling is not adding machines everywhere. Throughput is constrained by the narrowest effective boundary: source API quotas, network, partitions, shuffle, storage write rate, database concurrency, downstream service or even one serialized key. Scaling the wrong layer increases cost without increasing useful throughput.

Picture it in your head

A highway with eight lanes that narrows to one toll booth still processes cars at roughly the toll-booth rate. Widening the road before the booth does not solve the bottleneck.

Words before we use them

New word	Meaning before we use it
throughput	Useful work completed per unit time.
latency	Elapsed time for one unit/request/event to complete.
backpressure	Upstream slowing because downstream cannot keep up.
saturation	Resource operating near its practical capacity.
headroom	Unused capacity reserved for bursts, failures or growth.

Teacher explanation from first principles

Measure stages rather than guessing. Draw source -> ingest -> transform -> storage -> serving. For each, record capacity, observed throughput, latency, queue depth and limits. Apply simple capacity arithmetic: required throughput = daily/peak volume divided by useful processing window, adjusted for retries/headroom. Parallelism only helps when work can be partitioned and downstream accepts it.

A second architecture-review lens

Capacity planning should use percentiles and bursts, not only daily averages. A pipeline averaging 5k events/sec can still fail when a 10-minute burst hits 40k/sec. Include retry traffic and failure-mode capacity: if losing one partition or worker makes the remaining system exceed saturation, there is no real headroom. Observe queue depth and lag because they reveal when arrival rate exceeds service rate even before users see missed deadlines.

Useful video / source route

[VIDEO: Microsoft Learn - Apply Well-Architected Framework to your solution design - Open video](#)

Watch: 02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[OFFICIAL/CURRENT: arch](#)

[OFFICIAL/CURRENT: patterns](#)

Use rule: Watch/read for the mental model or current interface, then return to this lesson and its practice. External media is support; the architecture decision and evidence must be yours.

Follow with me once

A pipeline must process 36 million events in one hour: 10,000 events/sec average, but design for 2x headroom. Ingest supports 50k/sec, Spark supports 30k/sec, sink supports 12k/sec. Adding Spark executors cannot meet a 20k/sec target because sink is bottleneck. Options are sink batching/partitioning, a different serving design, a longer window or reduced write amplification.

What you should see / why it worked

A bottleneck table with measured/assumed capacities and the smallest change that raises end-to-end capacity. The important part is the trace from requirement to decision to evidence; the same product name can be correct in one

requirement set and wrong in another.

Break it: a realistic failure

A job runs slowly, so the team doubles compute. The source API is throttled at 500 requests/minute. Cloud cost doubles while duration barely changes.

Reason through a realistic incident

Preserve the evidence first. State the violated requirement, then identify the smallest failing boundary. Separate business-data state from compute/configuration/identity state. Bound the blast radius, choose a reversible repair, validate the original requirement, and finally test a changed case. Architecture recovery should reduce uncertainty, not hide it behind more components.

Try it yourself

Use the supplied capacity JSON. Calculate required average and burst throughput, identify bottleneck, and propose one scale-up and one architecture change.

Fresh changed retry

Simulate losing one worker/partition domain and keep 25% headroom. Recalculate whether the design still meets the processing window.

Validation and recovery ladder

- PASS only when your decision cites a requirement/constraint and an independent control or measurement.
- If a fact is unknown, label it as an assumption and state what you would measure before committing.
- Reject a design that solves the happy path but has no replay/recovery, ownership, security boundary or cost story.
- After repair/review, change latency, volume, budget or compliance and prove you can adapt instead of reciting the
- same diagram.

Revision in your own words

Scaling and bottleneck reasoning is a decision problem. Remember the requirement, the competing options, the trade-off, the failure boundary and the evidence that would validate the choice.

Teach it back

Why does scaling a non-bottleneck often produce almost no improvement?

Evidence to keep

Save your first architecture/scorecard, reviewer challenge, corrected decision, changed-case retry, and a short ADR explaining why you chose one option over another. Do not erase the first design when you improve it.

DE17-06

Reliability and failure domains

Watch first

[1. Microsoft Learn - Start improving the reliability of your Azure workloads](#)

00:58-03:58 reliability/platform vs application; 03:58-09:27 building blocks; 11:08 architecture walkthrough

[2. Microsoft Learn - Apply Well-Architected Framework to your solution design](#)

02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 6 of 10 | Primary practice:

16_M17/practice/architecture_patterns/06_failure_domain_budget.py

Why this matters

Reliable architecture assumes components fail. The design limits how far a failure spreads, preserves durable evidence, and defines recovery that can be tested. Availability is an end-to-end property: redundant compute does not help if every worker depends on one unrecoverable state store or one credential.

Picture it in your head

Ships have watertight compartments so one breach does not sink the whole vessel. Failure domains are architecture compartments: region, zone, cluster, account, database, partition or deployment boundary.

Words before we use them

New word	Meaning before we use it
failure domain	Set of components likely to fail together from one cause.
RTO	Target time to restore service after disruption.
RPO	Maximum acceptable data loss measured in time/state.
redundancy	Additional components/copies used so one failure need not stop service.
graceful degradation	Continuing a reduced but safe service instead of total failure or incorrect output.

Teacher explanation from first principles

Start from business impact, then choose RTO/RPO. Map state: raw source evidence, checkpoints, metadata, serving tables, secrets and code. Ask what happens if each domain disappears. Multi-zone compute without replicated state is false comfort. Replication also is not backup: corruption can replicate. Reliable data systems usually combine durable immutable inputs, idempotent/replayable processing, protected state/checkpoints, tested restore and controlled publish boundaries.

A second architecture-review lens

Reliability also requires correlated-failure thinking. Two copies in the same account, credential boundary, region, or deployment can fail together. Ask 'what common thing can kill both?' for every redundancy claim. Recovery plans should be exercised using known-good evidence: restore a checkpoint copy, replay a bounded raw range, verify key counts and business totals, then republish. A backup whose restore path is untested is only a hope, not proven recovery.

Useful video / source route

[VIDEO: Microsoft Learn - Start improving the reliability of your Azure workloads - Open video](#)

Watch: 00:58-03:58 reliability/platform vs application; 03:58-09:27 building blocks; 11:08 architecture walkthrough

[VIDEO: Microsoft Learn - Apply Well-Architected Framework to your solution design - Open video](#)

Watch: 02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[OFFICIAL/CURRENT: waf](#)

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch/read for the mental model or current interface, then return to this lesson and its practice. External media is support; the architecture decision and evidence must be yours.

Follow with me once

A daily pipeline has RTO 2 hours and RPO 0 for accepted raw files. Raw objects are immutable and replicated, transforms are replayable, curated publish is versioned, and last-known-good output remains available if validation fails. Compute can be recreated. The checkpoint state is backed up and tied to durable output so progress never outruns data.

What you should see / why it worked

A failure-domain table: failure, blast radius, detection, safe state, recovery action, RTO/RPO evidence. The important part is the trace from requirement to decision to evidence; the same product name can be correct in one requirement set and wrong in another.

Break it: a realistic failure

All services run across two zones, but credentials live in one manually maintained file and the only checkpoint is on one VM disk. The diagram looks redundant; actual recovery has single points of failure.

Reason through a realistic incident

Preserve the evidence first. State the violated requirement, then identify the smallest failing boundary. Separate business-data state from compute/configuration/identity state. Bound the blast radius, choose a reversible repair, validate the original requirement, and finally test a changed case. Architecture recovery should reduce uncertainty, not hide it behind more components.

Try it yourself

Threat-model a lakehouse pipeline for loss of one worker, one zone, catalog access, checkpoint store and an accidental bad deployment. Define safe behavior for each.

Fresh changed retry

Tighten RTO from 4 hours to 15 minutes while keeping RPO zero. Identify which design changes cost more and which risks cannot be removed merely by adding replicas.

Validation and recovery ladder

- PASS only when your decision cites a requirement/constraint and an independent control or measurement.
- If a fact is unknown, label it as an assumption and state what you would measure before committing.
- Reject a design that solves the happy path but has no replay/recovery, ownership, security boundary or cost story.
- After repair/review, change latency, volume, budget or compliance and prove you can adapt instead of reciting the
- same diagram.

Revision in your own words

Reliability and failure domains is a decision problem. Remember the requirement, the competing options, the trade-off, the failure boundary and the evidence that would validate the choice.

Teach it back

Explain the difference between replication, backup and replay. Why do you sometimes need all three?

Evidence to keep

Save your first architecture/scorecard, reviewer challenge, corrected decision, changed-case retry, and a short ADR explaining why you chose one option over another. Do not erase the first design when you improve it.

DE17-07

Cost-aware architecture

Watch first

[1. Microsoft Developer - Start optimizing your Azure costs](#)

01:05-02:10 common cost issues; 02:10-04:00 design/provision/monitor/optimize; 04:00 onward monitoring/optimization

[2. Microsoft Learn - Apply Well-Architected Framework to your solution design](#)

02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 7 of 10 | Primary practice: 16_M17/practice/architecture_patterns/07_cost_model.py

Why this matters

Cost is an architecture constraint, not a finance cleanup step. Data systems spend across storage, compute, data movement, managed-service premiums, observability, redundancy and people. Optimization means paying for requirements intentionally, not simply choosing the lowest unit price.

Picture it in your head

A cheap printer with expensive ink can cost more over its life than an expensive efficient printer. Architecture cost is total lifecycle cost and usage shape, not sticker price.

Words before we use them

New word	Meaning before we use it
TCO	Total cost of ownership including infrastructure, operations and people.
unit economics	Cost per useful unit such as TB processed, 1M events or dashboard query.
egress	Data transfer leaving a region/provider/service boundary that may incur cost.
right-sizing	Matching allocated capacity to real workload needs.
cost guardrail	Budget/usage rule that triggers review before uncontrolled spend continues.

Teacher explanation from first principles

Create a simple model before optimizing. Separate fixed and variable cost. Measure cost per business-relevant unit and by environment. Watch data movement: copying the same 20 TB between services can dominate cheap storage. Reliability has price - replicas and warm standby consume resources - but downtime also has business cost. The goal is to identify the cheapest architecture that satisfies the requirement set, not to delete reliability controls.

A second architecture-review lens

Cost reviews should protect unit economics from misleading totals. A growing bill can be healthy if business volume grows faster and cost per million events falls; a flat bill can hide waste if workload shrinks. Attribute spend to workloads/environments, keep non-production schedules deliberate, and model data movement. Human operating cost matters too: a technically cheap self-hosted cluster that consumes half an engineer can be more expensive than a managed service with higher infrastructure charges.

Useful video / source route

[VIDEO: Microsoft Developer - Start optimizing your Azure costs - Open video](#)

Watch: 01:05-02:10 common cost issues; 02:10-04:00 design/provision/monitor/optimize; 04:00 onward monitoring/optimization

[VIDEO: Microsoft Learn - Apply Well-Architected Framework to your solution design - Open video](#)

Watch: 02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[OFFICIAL/CURRENT: waf](#)

Use rule: Watch/read for the mental model or current interface, then return to this lesson and its practice. External media is support; the architecture decision and evidence must be yours.

Follow with me once

Option A is serverless query at 5 currency units/TB, scanning 40 TB/day = 200/day. Partitioning/pruning cuts scans to 8 TB/day = 40/day. Option B uses always-on cluster costing 120/day but handles predictable heavy transformations. Compare workload shape and operations, then model 30-day cost and sensitivity at 2x volume.

What you should see / why it worked

A cost model showing assumptions, unit costs, volume, reliability overhead and a sensitivity case. Cost claims are traceable rather than 'serverless is cheaper'. The important part is the trace from requirement to decision to evidence; the same product name can be correct in one requirement set and wrong in another.

Break it: a realistic failure

To save money a team removes duplicate raw retention and keeps only transformed outputs. A later bug cannot be replayed or audited. The immediate storage saving destroys recoverability and creates much larger incident cost.

Reason through a realistic incident

Preserve the evidence first. State the violated requirement, then identify the smallest failing boundary. Separate business-data state from compute/configuration/identity state. Bound the blast radius, choose a reversible repair, validate the original requirement, and finally test a changed case. Architecture recovery should reduce uncertainty, not hide it behind more components.

Try it yourself

Build a monthly model for storage, compute, egress and support labor for two architecture options in the supplied scenario. Identify top two cost drivers and one safe optimization for each.

Fresh changed retry

Double data volume and cut query frequency by half. Recompute rather than assuming your original winner stays cheapest.

Validation and recovery ladder

- PASS only when your decision cites a requirement/constraint and an independent control or measurement.
- If a fact is unknown, label it as an assumption and state what you would measure before committing.
- Reject a design that solves the happy path but has no replay/recovery, ownership, security boundary or cost story.
- After repair/review, change latency, volume, budget or compliance and prove you can adapt instead of reciting the
- same diagram.

Revision in your own words

Cost-aware architecture is a decision problem. Remember the requirement, the competing options, the trade-off, the failure boundary and the evidence that would validate the choice.

Teach it back

Why is cost optimization a trade-off discipline rather than simply reducing resource count?

Evidence to keep

Save your first architecture/scorecard, reviewer challenge, corrected decision, changed-case retry, and a short ADR explaining why you chose one option over another. Do not erase the first design when you improve it.

DE17-08

Security and governance by design

Watch first

[1. Microsoft Learn - Apply Well-Architected Framework to your solution design](#)

02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[2. HashiCorp - Introduction to Vault](#)

01:04-05:41 Vault goal, discovery/import/centralized access; 11:06-15:46 architecture; 18:45-20:10 auditing/policy

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 8 of 10 | Primary practice:

16_M17/practice/architecture_patterns/08_security_governance_matrix.py

Why this matters

Security added at the end becomes a patchwork of broad permissions and manual exceptions. Architecture should decide identity boundaries, least privilege, encryption, data classification, network paths, auditability, retention and ownership before data is widely copied.

Picture it in your head

A secure office building is designed with zones, badges, visitor routes, locked rooms and audit cameras. It is harder to retrofit those boundaries after everyone already has master keys.

Words before we use them

New word	Meaning before we use it
trust boundary	Point where identity, privilege or data exposure changes and requires explicit control.
defense in depth	Multiple independent controls so one failure does not immediately expose the asset.
data minimization	Collecting/retaining/exposing only data needed for the purpose.
policy as code	Machine-checkable rules stored and reviewed like code.
governance	Ownership, standards and controls that keep data usable, accountable and compliant.

Teacher explanation from first principles

Draw trust boundaries across source, ingestion identity, raw storage, transform identity, curated serving and consumers. Prefer workload identity/short-lived credentials over embedded secrets where supported. Separate admin, producer and consumer privileges. Classify sensitive columns early so masking/tokenization and retention happen before broad analytics access. Governance also needs ownership, catalog/lineage, quality contracts and exception processes.

A second architecture-review lens

Security architecture should include the failure path. During incidents, engineers often request broad emergency access. Define a controlled break-glass mechanism with approval, time limit and audit trail rather than normalizing permanent admin rights. Governance should also describe change: who can approve a schema change, who owns quality incidents, how sensitive classification propagates, and how deleted/retained data is proven. Security is an operating model as well as encryption settings.

Useful video / source route

[VIDEO: Microsoft Learn - Apply Well-Architected Framework to your solution design - Open video](#)

Watch: 02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[VIDEO: HashiCorp - Introduction to Vault - Open video](#)

Watch: 01:04-05:41 Vault goal, discovery/import/centralized access; 11:06-15:46 architecture; 18:45-20:10 auditing/policy

[OFFICIAL/CURRENT: waf](#)

[OFFICIAL/CURRENT: arch](#)

[OFFICIAL/CURRENT: vault](#)

Use rule: Watch/read for the mental model or current interface, then return to this lesson and its practice. External media is support; the architecture decision and evidence must be yours.

Follow with me once

Orders arrive with email and payment token. Ingest identity can write raw but not read curated. Transform identity reads raw and writes curated. Analytics users read masked curated tables; only an approved support role can access

restricted contact data. Secrets are retrieved from a managed secrets system; audit logs and lineage record access/change paths.

What you should see / why it worked

A security/governance matrix connecting persona -> data asset -> allowed action -> control -> evidence. No wildcard 'owner' role used as a shortcut. The important part is the trace from requirement to decision to evidence; the same product name can be correct in one requirement set and wrong in another.

Break it: a realistic failure

A pipeline fails with permission denied, so an engineer grants the service account subscription-owner access. The incident clears, but a small operational problem becomes a platform-wide security risk.

Reason through a realistic incident

Preserve the evidence first. State the violated requirement, then identify the smallest failing boundary. Separate business-data state from compute/configuration/identity state. Bound the blast radius, choose a reversible repair, validate the original requirement, and finally test a changed case. Architecture recovery should reduce uncertainty, not hide it behind more components.

Try it yourself

Create a trust-boundary diagram and RBAC table for a healthcare ingestion/lakehouse/reporting flow. Include restricted fields, encryption boundaries, audit evidence and deletion/retention responsibility.

Fresh changed retry

Add a third-party data processor and cross-region copy. Identify new trust/compliance boundaries before changing service choices.

Validation and recovery ladder

- PASS only when your decision cites a requirement/constraint and an independent control or measurement.
- If a fact is unknown, label it as an assumption and state what you would measure before committing.
- Reject a design that solves the happy path but has no replay/recovery, ownership, security boundary or cost story.
- After repair/review, change latency, volume, budget or compliance and prove you can adapt instead of reciting the
- same diagram.

Revision in your own words

Security and governance by design is a decision problem. Remember the requirement, the competing options, the trade-off, the failure boundary and the evidence that would validate the choice.

Teach it back

What is the difference between authentication, authorization and governance? Give one architecture control for each.

Evidence to keep

Save your first architecture/scorecard, reviewer challenge, corrected decision, changed-case retry, and a short ADR explaining why you chose one option over another. Do not erase the first design when you improve it.

DE17-09

Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes

Watch first

[1. HashiCorp - Introduction to Vault](#)

01:04-05:41 Vault goal, discovery/import/centralized access; 11:06-15:46 architecture; 18:45-20:10 auditing/policy

[2. Google Cloud Tech - Kubernetes architecture: Nodes and control plane](#)

00:17-03:02 node/control-plane mental model and automation

[3. Thoughtworks - Core principles of Data Mesh](#)

Optional visual - no verified bounded range; recordings cover domain ownership, data as product, self-service platform and federated governance

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 9 of 10 | Primary practice:

16_M17/practice/architecture_patterns/09_promotion_foundations_map.py

Why this matters

Job descriptions often mention CDC, data contracts, secrets platforms, data mesh and Kubernetes together. A junior engineer does not need five specialist certifications before applying, but should understand what problem each solves, where it fits, and what trade-offs or ownership it introduces. This lesson is a promotion map, not a replacement for dedicated platform engineering courses.

Picture it in your head

Think of a city map before learning every neighborhood. CDC is a way data changes travel, contracts define promises at borders, Vault protects secrets/identity material, data mesh changes ownership/product organization, and Kubernetes is a workload orchestration platform. Knowing the map prevents category mistakes.

Words before we use them

New word	Meaning before we use it
CDC	Change Data Capture: publishing/reading inserts, updates and deletes from source change history/logs.
data contract	Explicit producer-consumer agreement about schema, semantics, quality and evolution.
secrets manager	System controlling storage/issuance/rotation/access to sensitive credentials and keys.
data mesh	Sociotechnical approach emphasizing domain ownership, data as product, self-service platform and federated governance.
Kubernetes	Platform for orchestrating containerized workloads through declarative desired state and controllers.

Teacher explanation from first principles

CDC is useful when incremental source changes matter and polling/full reloads are inefficient or too slow; it still requires ordering, schema evolution, deletes and replay thinking. Data contracts make interface expectations reviewable before producers silently break consumers. Vault-type systems centralize secret lifecycle and identity/policy instead of scattering static credentials. Data mesh is mainly an operating/ownership model, not a database product. Kubernetes can standardize container scheduling, service discovery and desired-state operations but adds platform complexity; data engineers should not deploy it merely because a pipeline uses containers.

A second architecture-review lens

Use the promotion map to avoid resume-driven architecture. CDC does not remove the need for idempotent consumers; contracts do not stop every semantic misunderstanding; Vault does not automatically make an overprivileged application safe; data mesh does not eliminate central platform/governance needs; Kubernetes does not make a workload scalable by itself. Each concept shifts a specific problem and introduces its own ownership. Promotion readiness means you can discuss these boundaries clearly and know when deeper specialist study is required.

Useful video / source route

[VIDEO: HashiCorp - Introduction to Vault - Open video](#)

Watch: 01:04-05:41 Vault goal, discovery/import/centralized access; 11:06-15:46 architecture; 18:45-20:10 auditing/policy

[VIDEO: Google Cloud Tech - Kubernetes architecture: Nodes and control plane - Open video](#)

Watch: 00:17-03:02 node/control-plane mental model and automation

[VIDEO: Thoughtworks - Core principles of Data Mesh - Open video](#)

Watch: Optional visual - no verified bounded range; recordings cover domain ownership, data as product, self-service platform and federated governance

[OFFICIAL/CURRENT: debezium](#)

[OFFICIAL/CURRENT: datacontract](#)

[OFFICIAL/CURRENT: vault](#)

[OFFICIAL/CURRENT: datamesh](#)

[OFFICIAL/CURRENT: k8s](#)

Use rule: Watch/read for the mental model or current interface, then return to this lesson and its practice. External media is support; the architecture decision and evidence must be yours.

Follow with me once

An e-commerce company has PostgreSQL orders, a lakehouse, and many domain teams. Debezium-like CDC publishes order changes to Kafka; a data contract defines order-event keys, required fields and compatibility; workload identity/Vault supplies short-lived database credentials; the Orders domain owns the curated order data product; the platform team may run shared connectors/processors on managed Kubernetes. Each component has a different job. Removing one does not mean another automatically replaces it.

What you should see / why it worked

A five-column map: technology/concept, problem solved, where it sits, major risk/ownership, and when not to use it. You can explain each in 60 seconds without pretending production mastery. The important part is the trace from requirement to decision to evidence; the same product name can be correct in one requirement set and wrong in another.

Break it: a realistic failure

A team says 'we are doing data mesh, so we need Kubernetes and Kafka.' That confuses organizational/data-product principles with implementation choices. Mesh can use many technologies; Kubernetes does not create domain ownership or trustworthy data products.

Reason through a realistic incident

Preserve the evidence first. State the violated requirement, then identify the smallest failing boundary. Separate business-data state from compute/configuration/identity state. Bound the blast radius, choose a reversible repair, validate the original requirement, and finally test a changed case. Architecture recovery should reduce uncertainty, not hide it behind more components.

Try it yourself

For a bank change-data pipeline, place CDC, contract, secrets, domain ownership and workload orchestration on an architecture diagram. Mark which are required by the scenario and which are optional.

Fresh changed retry

Remove real-time requirements and move to daily file exchange with a small team. Re-evaluate which five concepts remain useful and which would be unjustified complexity.

Validation and recovery ladder

- PASS only when your decision cites a requirement/constraint and an independent control or measurement.
- If a fact is unknown, label it as an assumption and state what you would measure before committing.
- Reject a design that solves the happy path but has no replay/recovery, ownership, security boundary or cost story.
- After repair/review, change latency, volume, budget or compliance and prove you can adapt instead of reciting the
- same diagram.

Revision in your own words

Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes is a decision problem. Remember the requirement, the competing options, the trade-off, the failure boundary and the evidence that would validate the choice.

Teach it back

In one sentence each, distinguish CDC, a data contract, Vault, data mesh and Kubernetes. Then name one dangerous category mistake.

Evidence to keep

Save your first architecture/scorecard, reviewer challenge, corrected decision, changed-case retry, and a short ADR explaining why you chose one option over another. Do not erase the first design when you improve it.

DE17-10

Architecture review and defense

Watch first

[1. Microsoft Learn - Apply Well-Architected Framework to your solution design](#)

02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[2. Microsoft Learn - Start improving the reliability of your Azure workloads](#)

00:58-03:58 reliability/platform vs application; 03:58-09:27 building blocks; 11:08 architecture walkthrough

[3. Microsoft Developer - Start optimizing your Azure costs](#)

01:05-02:10 common cost issues; 02:10-04:00 design/provision/monitor/optimize; 04:00 onward monitoring/optimization

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 10 of 10 | Primary practice:

16_M17/practice/architecture_patterns/10_architecture_review_defense.py

Why this matters

Architecture interviews and real design reviews are not drawing competitions. Reviewers want to see whether you can explain requirements, boundaries, failure behavior, security, cost, alternatives and evidence. A defensible architecture makes assumptions visible and survives changed conditions without collapsing into 'because this tool is popular'.

Picture it in your head

A courtroom defense is structured around evidence and reasoning. You must state the claim, support it, acknowledge weaknesses and explain why alternatives were rejected. Architecture defense is similar, but the goal is collaborative risk reduction rather than winning an argument.

Words before we use them

New word	Meaning before we use it
ADR	Architecture Decision Record: concise record of context, options, decision and consequences.
assumption	Belief used by the design that needs validation or a safety margin.
risk register	List of important failure/uncertainty items, likelihood/impact and mitigation.
fitness function	Automated/measurable check that tests an architecture property over time.
review gate	Point where evidence must be accepted before promotion/release.

Teacher explanation from first principles

Use a repeatable review order: 1 requirements/constraints, 2 data flow and grain, 3 batch/stream/storage choices, 4 failure/replay/state, 5 security/governance, 6 scaling/cost, 7 observability/operations, 8 alternatives/trade-offs, 9 rollout/migration, 10 evidence. Do not hide uncertainty. A strong answer says 'I would measure X before choosing Y' when the required fact is unknown.

A second architecture-review lens

A mature review explicitly records rejected options. If you choose managed batch, state why streaming or self-hosting lost under the present requirements. Then define a trigger that would reopen the decision. This protects the team from cargo-cult architecture and from treating an old ADR as eternal. In interviews, saying 'I would validate peak rate and consumer latency before finalizing partition count' is stronger than inventing a confident number with no evidence.

Useful video / source route

[VIDEO: Microsoft Learn - Apply Well-Architected Framework to your solution design - Open video](#)

Watch: 02:35-07:53 requirements; 09:16-11:41 trade-offs; 11:41-33:10 reliability; 33:10-43:16 performance; 43:16-52:01 security; 57:40-1:02:44 cost

[VIDEO: Microsoft Learn - Start improving the reliability of your Azure workloads - Open video](#)

Watch: 00:58-03:58 reliability/platform vs application; 03:58-09:27 building blocks; 11:08 architecture walkthrough

[VIDEO: Microsoft Developer - Start optimizing your Azure costs - Open video](#)

Watch: 01:05-02:10 common cost issues; 02:10-04:00 design/provision/monitor/optimize; 04:00 onward monitoring/optimization

[OFFICIAL/CURRENT: arch](#)

[OFFICIAL/CURRENT: waf](#)

[OFFICIAL/CURRENT: patterns](#)

Use rule: Watch/read for the mental model or current interface, then return to this lesson and its practice. External media is support; the architecture decision and evidence must be yours.

Follow with me once

Defend a regional retail lakehouse: hourly batch ingestion, immutable raw, Delta silver/gold, warehouse/SQL serving, managed orchestration, least-privilege identities, freshness/reconciliation gates, last-known-good publish, cost budgets and replay. Reviewers challenge: why not streaming? why not only warehouse? what if region fails? what if volume 5x? The defense returns to the requirement table and quantified trade-offs.

What you should see / why it worked

A one-page architecture brief, diagram, ADRs and 5-minute verbal defense where each major component has a requirement-based reason and a failure/recovery story. The important part is the trace from requirement to decision to evidence; the same product name can be correct in one requirement set and wrong in another.

Break it: a realistic failure

The presenter answers every challenge by naming another service. The diagram grows but requirements, costs and ownership remain unclear. Complexity is being used to hide uncertainty instead of resolving it.

Reason through a realistic incident

Preserve the evidence first. State the violated requirement, then identify the smallest failing boundary. Separate business-data state from compute/configuration/identity state. Bound the blast radius, choose a reversible repair, validate the original requirement, and finally test a changed case. Architecture recovery should reduce uncertainty, not hide it behind more components.

Try it yourself

Prepare the supplied architecture-review case. Produce requirement table, diagram, three ADRs, failure-domain table, cost estimate, security matrix and a 5-minute defense. Record five questions you expect from reviewers.

Fresh changed retry

Use the fresh Gate-B-style case with different latency, budget and compliance constraints. Defend a changed architecture without copying your first diagram.

Validation and recovery ladder

- PASS only when your decision cites a requirement/constraint and an independent control or measurement.
- If a fact is unknown, label it as an assumption and state what you would measure before committing.
- Reject a design that solves the happy path but has no replay/recovery, ownership, security boundary or cost story.
- After repair/review, change latency, volume, budget or compliance and prove you can adapt instead of reciting the
- same diagram.

Revision in your own words

Architecture review and defense is a decision problem. Remember the requirement, the competing options, the trade-off, the failure boundary and the evidence that would validate the choice.

Teach it back

Give your architecture defense in this order: requirement, decision, evidence, trade-off, failure behavior. Why is that stronger than describing components left-to-right?

Evidence to keep

Save your first architecture/scorecard, reviewer challenge, corrected decision, changed-case retry, and a short ADR explaining why you chose one option over another. Do not erase the first design when you improve it.